

Module 1: Introduction to Amazon Web Services

Cloud Deployment Models: Cloud-Based vs. On-Premises vs. Hybrid

Criteria	Cloud-Based	On-Premises (Private Cloud)	Hybrid
Core Purpose	Run everything on public cloud; no local hardware	Full control; run infra in own data center	Mix cloud + on-prem; bridge both worlds
Key Characteristic	Fully managed; highly elastic	Max control & compliance; fixed capacity	Flexible; legacy + cloud coexist
Cost Structure	Pure OpEx; pay-as-you-go	High CapEx; ongoing maintenance	Mixed CapEx + OpEx
Main Use Case	New apps; startups; scalable workloads	Strict regulatory/data sovereignty needs	Migrating legacy apps; regulated industries

Cost Models: Upfront Expense (CapEx) vs. Variable Expense (OpEx)

Criteria	CapEx (Upfront)	OpEx (Variable)
Core Purpose	Buy & own physical resources in advance	Pay only for what you consume, when you use it
Key Characteristic	Fixed asset; depreciated over time	Flexible; scales with usage; no ownership
Cost Structure	Large initial investment; predictable but rigid	No upfront cost; ongoing variable spend
Main Use Case	Stable, predictable long-term workloads on-prem	Cloud workloads; variable or unpredictable demand

Module 2: Compute in the Cloud

EC2 Instance Types: General Purpose vs. Compute Optimized vs. Memory Optimized vs. Accelerated Computing vs. Storage Optimized

Criteria	General Purpose	Compute Optimized	Memory Optimized	Accelerated Computing	Storage Optimized
Core Purpose	Balanced CPU/RAM/network	High CPU performance	Large in-memory datasets	GPU/hardware accelerator tasks	High I/O to local storage
Key Characteristic	Balanced ratio	High CPU-to-memory ratio	High RAM-to-CPU ratio	FPGAs / GPUs	High sequential read/write IOPS
Cost Structure	Moderate; versatile baseline	Higher; CPU-intensive	Higher; large RAM	Highest; specialized hardware	Moderate-high; NVMe SSDs
Main Use Case	Web servers, small DBs, dev/test	Batch processing, HPC, gaming servers	In-memory DBs (SAP HANA), real-time analytics	ML inference, video rendering, scientific computing	Data warehousing, log/search workloads

EC2 Pricing Models: On-Demand vs. Savings Plans vs. Reserved Instances vs. Spot Instances vs. Dedicated Hosts

Criteria	On-Demand	Savings Plans	Reserved Instances	Spot Instances	Dedicated Hosts
----------	-----------	---------------	--------------------	----------------	-----------------

Core Purpose	Pay per second/hour, no commitment	Commit to \$/hr spend; flexible usage	Reserve capacity for 1–3 yrs	Use spare AWS capacity at deep discount	Physical server; no instance sharing
Key Characteristic	No commitment; full flexibility	Up to 72% off; flexible across services	Up to 75% off; specific instance type/region	Up to 90% off; can be interrupted	Full host control; bring-your-own-license
Cost Structure	Highest per-unit rate	Hourly commitment; predictable savings	Upfront/partial/no upfront options	Lowest cost; unpredictable availability	Most expensive; compliance/licensing driven
Main Use Case	Short-term, spiky, or unpredictable workloads	Steady-state workloads needing flexibility	Known, steady-state workloads (1–3 yr horizon)	Fault-tolerant batch jobs, big data, CI/CD	Regulatory compliance; per-socket/VM licensing

Scaling Methods: Vertical Scaling (Scaling Up) vs. Horizontal Scaling (Scaling Out)

Criteria	Vertical Scaling (Scale Up)	Horizontal Scaling (Scale Out)
Core Purpose	Add more CPU/RAM to existing instance	Add more instances to share load
Key Characteristic	Single larger machine; has an upper limit	Multiple machines; theoretically unlimited
Cost Structure	Downtime often required; simple but limited	No downtime; more complex (load balancer needed)
Main Use Case	Legacy apps that can't distribute; quick wins	Cloud-native apps; high availability; web tiers

Auto Scaling Approaches: Dynamic Scaling vs. Predictive Scaling

Criteria	Dynamic Scaling	Predictive Scaling
Core Purpose	React to real-time demand changes	Proactively schedule capacity before demand hits
Key Characteristic	Reactive; triggered by CloudWatch metrics	ML-based; analyzes historical load patterns
Cost Structure	Efficient; may lag behind sudden spikes	Efficient; avoids cold-start lag; needs history
Main Use Case	Unpredictable, variable traffic patterns	Recurring, predictable load (e.g., business hours)

Application Architecture: Monolithic Applications vs. Microservices

Criteria	Monolithic	Microservices
Core Purpose	Single unified codebase; all components tightly coupled	Loosely coupled independent services; each does one thing
Key Characteristic	Simpler initially; single point of failure	Resilient; one service fails without affecting others
Cost Structure	Lower dev complexity upfront; hard to scale selectively	Higher ops complexity; scale individual components cheaply
Main Use Case	Small apps, MVPs, simple domains	Large-scale cloud apps, independent team ownership

Compute Categories: Virtual Servers (EC2) vs. Serverless (Lambda) vs. Containers (ECS/EKS)

Criteria	EC2 (Virtual Servers)	Lambda (Serverless)	ECS/EKS (Containers)
----------	-----------------------	---------------------	----------------------

Core Purpose	Full VM control; configurable OS/runtime	Event-driven; run code without managing servers	Portable, consistent environments via containers
Key Characteristic	Full control; you manage OS/patches	Auto-scales to zero; max 15 min execution	Consistent across envs; orchestrated workloads
Cost Structure	Pay per hour even when idle	Pay per invocation + duration; very cost-efficient at low scale	Pay for underlying compute (EC2 or Fargate)
Main Use Case	Long-running apps, DBs, legacy lift-and-shift	Event processing, APIs, scheduled tasks, short jobs	Microservices, batch jobs, hybrid portability needs

Container Platforms: EC2 vs. AWS Fargate (for hosting containers)

Criteria	EC2 (for Containers)	AWS Fargate
Core Purpose	You manage underlying EC2 instances for containers	Serverless compute engine; AWS manages all infra
Key Characteristic	Full control; you patch/scale EC2 cluster	No servers to manage; auto-provisions per task
Cost Structure	Can be cheaper at scale; more operational work	Higher per-unit cost; zero ops overhead
Main Use Case	When you need GPU, custom AMIs, or max cost control	Teams wanting to run containers with no infra management

Messaging Services: Amazon SQS vs. Amazon SNS

Criteria	SQS (Simple Queue Service)	SNS (Simple Notification Service)
Core Purpose	Message queue; decouple producers & consumers	Pub/sub; broadcast messages to multiple subscribers
Key Characteristic	Pull-based; messages persist until consumed	Push-based; fan-out to many endpoints simultaneously
Cost Structure	Per request + data transfer	Per publish + per delivery
Main Use Case	Async task queues, load leveling, order processing	Notifications, alerts, triggering multiple services at once

Module 3: Global Infrastructure and Reliability

Infrastructure Components: Regions vs. Availability Zones vs. Edge Locations

Criteria	Regions	Availability Zones (AZs)	Edge Locations
Core Purpose	Geographic area containing multiple AZs	Isolated data center(s) within a Region	CDN endpoints; cache content close to users
Key Characteristic	Fully isolated; choose for latency/compliance	Connected via low-latency links; fault-isolated	Part of CloudFront; globally distributed (200+)
Cost Structure	Data transfer costs between Regions	Low-latency links within same Region	Reduces origin load; may add CloudFront cost
Main Use Case	Data sovereignty, DR, global deployment	High availability within a Region	Static asset caching, video streaming, low latency

Interaction Tools: AWS Management Console vs. AWS CLI vs. AWS SDKs

Criteria	Management Console	AWS CLI	AWS SDKs
Core Purpose	Web GUI for manual AWS interaction	Command-line tool for scripting AWS tasks	Programmatic API access from code
Key Characteristic	Point-and-click; easy for beginners	Scriptable; automatable; JSON/text output	Language-native (Python, Java, JS, etc.)
Cost Structure	Free; no extra cost	Free; no extra cost	Free; no extra cost
Main Use Case	Exploration, learning, one-off tasks	Automation scripts, CI/CD pipelines, bulk ops	App-level AWS integration, building cloud-native software

Managed Provisioning: AWS Elastic Beanstalk vs. AWS CloudFormation

Criteria	Elastic Beanstalk	AWS CloudFormation
Core Purpose	PaaS; deploy apps without managing infra	IaC; provision any AWS resource via templates
Key Characteristic	Opinionated; handles EC2, LB, scaling automatically	Flexible; full control via JSON/YAML templates
Cost Structure	Free (pay only for underlying resources)	Free (pay only for provisioned resources)
Main Use Case	Developers who want fast deploys with minimal infra config	DevOps teams needing repeatable, version-controlled infra

Module 4: Networking

Subnet Types: Public Subnets vs. Private Subnets

Criteria	Public Subnet	Private Subnet
Core Purpose	Resources accessible from/to the internet	Resources isolated from public internet
Key Characteristic	Has route to Internet Gateway (IGW)	No IGW route; internet via NAT Gateway only (outbound)
Cost Structure	Standard compute + public IP costs	Add NAT Gateway cost for outbound internet
Main Use Case	Web servers, load balancers, bastion hosts	Databases, app servers, internal services

Connectivity Options: VPN vs. AWS Direct Connect

Criteria	Virtual Private Network (VPN)	AWS Direct Connect
Core Purpose	Encrypted tunnel over public internet to AWS	Dedicated private physical connection to AWS
Key Characteristic	Fast to set up; variable latency; encrypted	Consistent low latency; not encrypted by default; 1–100 Gbps
Cost Structure	Low cost; pay per VPN connection + data	Higher cost; port hours + data transfer
Main Use Case	Quick hybrid setup, backup connectivity, small data volumes	High-throughput, low-latency, compliance-sensitive workloads

Network Security Layers: NACLs vs. Security Groups

Criteria	NACLs (Network ACLs)	Security Groups
----------	----------------------	-----------------

Core Purpose	Subnet-level firewall; first line of defense	Instance-level firewall; controls EC2/resource traffic
Key Characteristic	Stateless; evaluates inbound & outbound separately	Stateful; return traffic auto-allowed
Cost Structure	Free; part of VPC	Free; part of VPC
Main Use Case	Block/allow entire IP ranges at subnet level; add defense-in-depth	Fine-grained port/protocol rules per resource

Packet Filtering Logic: Stateless Filtering (NACLs) vs. Stateful Filtering (Security Groups)

Criteria	Stateless (NACLs)	Stateful (Security Groups)
Core Purpose	Each packet evaluated independently; no memory	Tracks connection state; return traffic auto-permitted
Key Characteristic	Must explicitly allow both inbound AND outbound rules	Allow inbound → return outbound automatically allowed
Cost Structure	More rules needed; harder to manage	Simpler rules; less operational overhead
Main Use Case	When you need strict bidirectional control at subnet level	Typical instance-level access control (web, DB ports)

Module 5: Storage and Databases

Block-Level Storage: Instance Stores vs. Amazon EBS

Criteria	Instance Store	Amazon EBS
Core Purpose	Ephemeral local disk physically attached to host	Persistent block storage independent of EC2 lifecycle
Key Characteristic	Fastest I/O; data lost when instance stops/terminates	Durable; persists independently; snapshots to S3
Cost Structure	Included in instance price; no extra cost	Additional cost per GB + IOPS provisioned
Main Use Case	Temp data, caches, buffers, scratch space	OS volumes, databases, any data needing persistence

Storage Technologies: Block Storage (EBS) vs. Object Storage (S3) vs. File Storage (EFS)

Criteria	Block Storage (EBS)	Object Storage (S3)	File Storage (EFS)
Core Purpose	Low-level disk blocks; mount as drive	Store any data as objects via HTTP API	Shared file system; mountable by many instances
Key Characteristic	Low latency; single AZ attach (usually)	Unlimited scale; 11 9s durability; region-wide	POSIX-compliant; auto-scales; multi-AZ
Cost Structure	Per GB provisioned + IOPS	Per GB stored + requests + retrieval	Per GB used (no provisioning)
Main Use Case	EC2 OS/DB volumes; low-latency random I/O	Backups, media, logs, data lakes, static web assets	Shared content between many EC2 instances; CMS, ML training data

AWS Shared Storage: Amazon EBS vs. Amazon EFS

Criteria	Amazon EBS	Amazon EFS
Core Purpose	Block storage; attach to a single EC2 instance	Managed NFS; mount across many instances simultaneously
Key Characteristic	Single-attach (one AZ); manual snapshots for backup	Multi-attach; grows/shrinks automatically; multi-AZ
Cost Structure	Pay for provisioned capacity	Pay for data stored (no provisioning)
Main Use Case	Single-instance workloads: databases, OS drives	Shared workloads: web farms, containerized apps, big data

Amazon S3 Storage Classes: S3 Standard vs. S3 Standard-IA vs. S3 One Zone-IA vs. S3 Intelligent-Tiering vs. S3 Glacier vs. S3 Glacier Deep Archive

Criteria	S3 Standard	S3 Standard-IA	S3 One Zone-IA	S3 Intelligent-Tiering	S3 Glacier	S3 Glacier Deep Archive
Core Purpose	Frequent access; high durability	Infrequent access; lower storage cost	IA in single AZ; cheaper but less resilient	Auto-moves data between tiers by access pattern	Long-term archive; retrieval in minutes–hours	Lowest-cost archive; retrieval in hours–12 hrs
Key Characteristic	11 9s durability; multi-AZ; millisecond access	Same durability as Standard; retrieval fee applies	One AZ only; 20% cheaper than Standard-IA; retrieval fee	Monitoring fee per object; no retrieval fees within tiers	Multi-AZ; configurable retrieval speed (expedited/standard/bulk)	Multi-AZ; cheapest storage; 12–48 hr retrieval
Cost Structure	Highest storage cost/GB; no retrieval fee	Lower storage; per-GB retrieval fee; min 30-day charge	Lowest IA cost; retrieval fee; min 30-day charge	Small monitoring fee; tiered storage savings; no retrieval fees	Very low storage; retrieval fees vary by speed	Lowest storage cost; retrieval fees; min 180-day charge
Main Use Case	Active content, websites, mobile apps	Backups, DR files accessed < once/month	Reproducible data (e.g., thumbnails); non-critical IA	Unknown or changing access patterns; automated optimization	Compliance archives, media preservation, 1–7 yr retention	Regulatory archives, tape replacement, 7+ yr retention

Database Types: Relational Databases (RDS/Aurora) vs. Nonrelational / NoSQL (DynamoDB)

Criteria	Relational DB (RDS / Aurora)	NoSQL DB (DynamoDB)
Core Purpose	Structured data with fixed schema & relationships	Flexible schema; key-value or document model
Key Characteristic	SQL; ACID transactions; joins; strict schema	Millisecond latency at any scale; schema-less; horizontal scale
Cost Structure	Instance-based pricing; storage + I/O	Pay per read/write capacity or on-demand; storage
Main Use Case	ERP, CRM, e-commerce, complex relational queries	Gaming, IoT, session stores, real-time apps, high-velocity data

Database Use Cases: OLTP (RDS) vs. OLAP / Big Data (Amazon Redshift)

Criteria	OLTP – RDS / Aurora	OLAP – Amazon Redshift
Core Purpose	Handle many small, fast transactions in real-time	Analyze large volumes of historical data
Key Characteristic	Row-based storage; optimized for inserts/updates/deletes	Columnar storage; optimized for aggregations & large scans
Cost Structure	Instance + storage; scales vertically	Cluster-based; can use Serverless; scales by nodes
Main Use Case	Order processing, banking, inventory updates	Business intelligence, dashboards, data warehousing

Database Migration: Homogeneous Migration vs. Heterogeneous Migration

Criteria	Homogeneous Migration	Heterogeneous Migration
Core Purpose	Migrate between same DB engines (e.g., MySQL → MySQL)	Migrate between different DB engines (e.g., Oracle → Aurora)
Key Characteristic	Simpler; schema/code compatible; use AWS DMS directly	Requires schema conversion (AWS SCT) before migration
Cost Structure	Lower effort; DMS only	Higher effort; SCT + DMS; possible app code changes
Main Use Case	Lift-and-shift same engine from on-prem to RDS	Re-platforming to save cost or adopt open-source engines

Caching Solutions: Amazon ElastiCache vs. DynamoDB Accelerator (DAX)

Criteria	Amazon ElastiCache	DynamoDB Accelerator (DAX)
Core Purpose	In-memory cache layer for any database or app	Purpose-built in-memory cache exclusively for DynamoDB
Key Characteristic	Supports Redis & Memcached; general-purpose	Fully managed; microsecond reads; DynamoDB API-compatible
Cost Structure	Node-based pricing (Redis/Memcached cluster)	Node-based pricing; additional to DynamoDB costs
Main Use Case	Cache RDS queries, session data, leaderboards; multi-DB scenarios	High-read DynamoDB workloads (gaming, auctions, flash sales)